

Министерство образования и науки Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа радиотехники и компьютерных технологий
Кафедра микропроцессорных технологий в интеллектуальных системах управления

Выпускная квалификационная работа магистра

Изменение трасс исполнения для модификации записанных образов памяти

Автор:

Студент 407 группы
Матренин Василий Николаевич

Научный руководитель:

Петушков Игорь Вячеславович

Научный консультант:

Матвеев Сергей Владимирович



Москва 2026

Аннотация

Трасса исполнения (далее – трасса) представляет собой упорядоченный набор записей о событиях, происходивших в ходе трассировочного запуска приложения [39; 53]. Подобные трассы широко применяются в индустрии для решения различных задач, таких как анализ производительности платформ, отладка, сбор профилей исполнения [15; 39; 54]. В рамках данной работы рассмотрены трассы для RISC-архитектуры.

Целевые сценарии исполнения, записанные в виде трасс, могут применяться для анализа поведения разрабатываемых платформ. При этом настройки системы памяти новой платформы (далее – новая конфигурация или новые настройки) могут отличаться от настроек платформы, на которой были записаны трассы. Возникает необходимость адаптации трасс под новую конфигурацию системы памяти.

Целью данной работы является разработка решения, позволяющего изменять трассы исполнения для модификации записанных в них образов памяти приложений в соответствии с ограничениями новой конфигурации системы памяти.

Представленное решение осуществляет перенос областей образа памяти в соответствии с новыми ограничениями посредством поиска и модификации адресов и смещений в трассе. Для генерации необходимых модификаций анализируется граф потока данных трассы. Рассмотрены алгоритмы сбора графа потока данных из трассы исполнения, поиска адресов и смещений в трассе с помощью обходов графа, обработки граничных случаев с помощью симуляционного прохода по модифицированной трассе. Описанный подход позволяет обеспечить консистентность потоков управления и данных в результирующих трассах, а также снизить влияние модификаций на репрезентативность этих трасс. Представленные алгоритмы могут быть использованы для других задач, связанных с модификацией адресов и смещений в трассах исполнения.

Временная и пространственная сложность обработки трассы линейна по числу инструкций и связей по данным между ними. Предложена метрика для предварительной оценки влияния производимых модификаций на репрезентативность трасс. Рассмотрены другие метрики для количественной оценки свойств реализации. Представлены результаты экспериментальных запусков разработанного средства на наборе целевых трасс заказчика для одной из конфигураций системы памяти. Описаны возможности для дальнейшего улучшения разработанного средства и новые направления его применения.

Предложенное решение успешно применяется для анализа поведения разрабатываемой системы на целевых сценариях исполнения и конфигурациях системы памяти.

Содержание

Аббревиатуры	5
Математические обозначения	5
Словарь терминов	5
1 Введение	7
1.1 Трассы исполнения	7
1.2 Репрезентативность трасс	9
1.3 Виртуальное адресное пространство	10
1.4 Цель работы	11
2 Обзор существующих решений	12
2.1 Позиционно-независимый код	12
2.2 Компоновщики	12
2.3 Компиляторы	13
2.4 Сборщики мусора ВМ	14
2.5 Динамическое восстановление вырезанных таблиц перемещений	14
2.6 Недостатки существующих решений	15
3 Решение задачи	16
3.1 Архитектура решения	16
3.2 Генерация карты перемещений	17
3.3 Сбор графа потока данных	18
3.4 Генерация модификаций трассы	22
3.4.1 Поиск адресов и смещений	24
3.4.2 Симуляционный проход по модифицированной трассе	26
3.5 Запись модифицированной трассы	27
3.6 Симуляция по трассе	27
3.7 Тестирование	29
3.8 Оценка решения	33
3.8.1 Пространственная и временная сложность	33
3.8.2 Оценка репрезентативности трасс	34
3.8.3 Профилировка решения по памяти	35

3.8.4	Тестовые запуски на целевых трассах	35
3.9	Возможные направления дальнейшего развития решения	38
3.9.1	Поддержка граничных случаев	38
3.9.2	Оптимизация по памяти	39
3.9.3	Поддержка векторных регистров	40
3.9.4	Модификация защищенных адресов	40
4	Заключение	42

Аббревиатуры

ASLR Address Space Layout Randomization, рандомизация адресного пространства. 12

GOT Global Offset Table, глобальная таблица смещений. 12

PIC Position-Independent Code, позиционно-независимый код. 12

PLT Procedure Linkage Table, таблица связывания процедур. 12

ВМ Виртуальная машина языка программирования. 14

ГПД Граф Потока Данных. Вершины графа моделируют инструкции трассы. Ребра графа задают зависимости по данным между инструкциями: значение, вычисленное или переданное одной инструкцией, используется другой инструкцией. С каждым ребром ассоциируются передаваемое значение и локация, в которой это значение находилось в момент трассировки, например регистр или область памяти. Термины вершина, динамическая инструкция и, если это не приводит к неоднозначности, инструкция используются взаимозаменяемо.. 16

ОС Операционная система. 8

ПО Программное обеспечение. 29

Математические обозначения

$G = (V, E)$ ГПД, где: V – множество вершин графа, E – множество ребер графа. 18

$\mathcal{P}(E)$ Множество всех подмножеств ребер ГПД. 18

Словарь терминов

Адресное ребро Ребро ГПД, значение которого используется целевой инструкцией в качестве адреса или считается адресом по другим косвенным признакам. 24

Инструкция приложения (статическая инструкция) Статическая машинная инструкция, расположенная в памяти трассируемой программы. 7

Инструкция трассы (динамическая инструкция) Записи трассы исполнения, отвечающие одному выполнению инструкции приложения. Далее, если не оговорено иное, под инструкцией понимается именно динамическая инструкция. 7

Исходное адресное ребро Ребро ГПД, на котором заканчивается построение обратного пути из адресного ребра в алгоритме поиска адресов. Значение этого ребра рассматривается как исходное значение, из которого формируется адрес. 25

Модуль Библиотека или приложение. 12

Модульное тестирование Тестирование группы связанных компонентов, образующих законченный модуль системы. Проверяет корректность взаимодействия внутри модуля и его интерфейсов. 29

Псевдо-инструкция Служебная вершина-исток ГПД, не отвечающая реальной инструкции трассы. Используется для отображения зависимостей, не порождаемых отдельными инструкциями, или для добавления вспомогательных зависимостей. 18

Сквозное тестирование (end-to-end testing) Тестирование полного бизнес-сценария в условиях, максимально приближенных к реальным. Проверяет работу всех компонентов системы вместе, включая внешние зависимости. 29

Трасса исполнения Набор упорядоченных записей о событиях, происходивших во время трассировочного запуска приложения. В рамках данной работы рассмотрены трассы для RISC-архитектуры. 7

Юнит тестирование Тестирование минимального фрагмента кода (функции или метода) в изоляции от других компонентов. 29

1 Введение

1.1 Трассы исполнения

Трасса исполнения представляет собой упорядоченный набор записей о событиях: исполнениях инструкций, операциях с памятью, системных вызовах, сигналах и т. д., происшедших во время трассировочного запуска приложения.

Трассы исполнения применяются в индустрии для различных задач, таких как: микроархитектурный и макро анализ приложений, сбор профилей исполнения, отладка [7; 14; 39; 40; 54]. В рамках данной работы рассмотрены трассы для RISC-архитектуры.

Трасса может содержать записи для нескольких потоков исполнения, однако для простоты в дальнейшем будем рассматривать только трассы исполнения с одним потоком. Описанное в данной работе решение нетрудно обобщается на трассы с произвольным числом потоков.

Будем называть инструкцией приложения (статической инструкцией) статическую машинную инструкцию, расположенную в памяти трассируемой программы. Записи трассы исполнения, отвечающие одному выполнению инструкции приложения назовем инструкцией трассы (динамической инструкцией). Далее, если не оговорено иное, под инструкцией понимается именно динамическая инструкция.

Рассматриваемые трассы исполнения содержат следующие основные виды записей:

- Начальное регистровое состояние

Содержит значения регистров общего назначения, векторных регистров и условных флагов в момент начала трассировки. Также такие записи используются для отображения состояния машины при переключениях контекста исполнения (обработка сигналов, изменение уровня привилегий).

- Исполнение инструкции

Содержит информацию об исполнении инструкции: адрес, кодировку и номер инструкции в трассе. С данной записью могут быть ассоциированы обращения в память и записи в регистры (см. ниже).

- Обращение в память

Содержит адрес, размер, данные и тип обращения (запись или чтение). Одна инструкция может иметь несколько ассоциированных с ней обращений к памяти. Данные для чтения из памяти соответствуют состоянию памяти до исполнения инструкции.

- Запись в регистр общего назначения

Содержит идентификатор регистра и его значение **после** исполнения инструкции. Одна инструкция может иметь несколько ассоциированных с ней записей в регистры. Такие записи используются для отслеживания изменений регистров общего назначения системными (системные вызовы, чтения системных регистров) и другими инструкциями.

Изменения памяти, осуществляемые операционной системой (ОС) или другими потоками исполнения будут отражены в трассе (в записях для рассматриваемого потока) только косвенно: при чтении памяти может быть прочитано новое значение, не соответствующее образу памяти, который наблюдался ранее. На рис. 1 приведен пример трассы исполнения, в том числе иллюстрирующий данную особенность (см. записи 2 и 6).

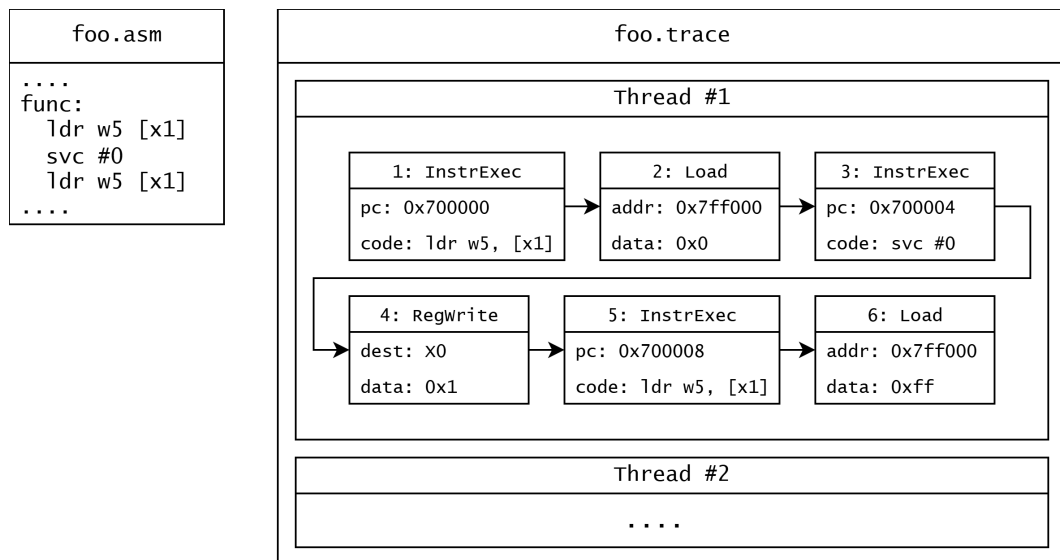


Рис. 1: Пример трассы исполнения.

При анализе трасс исполнения часто используются симуляторы [1; 48]. В таких случаях в состояние симулятора пробрасываются данные из трассы, для рассматриваемых трасс это: начальное регистровое состояние, коды инструкций, данные для чтений из памяти, новые значения регистров общего назначения. При этом зачастую потоки управления и данных симулятора проверяются на соответствие трассе для детектирования ошибок симуляции или невалидных трасс. Такой подход называется симуляцией по трассе.

Описанные особенности трасс исполнения и их обработки позволяют существенно модифицировать записанные сценарии и получать валидные трассы для дальнейшей обработки: можно изменить или добавить необходимые записи для коррекции потоков управления и данных. На рис. 2 приведен пример модифицированной трассы исполнения. Измененные данные выделены красным цветом.

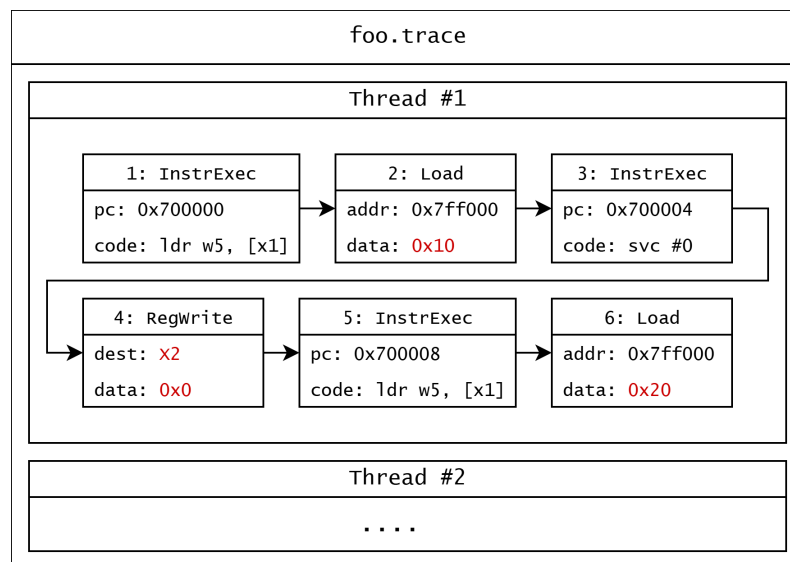


Рис. 2: Пример модифицированной трассы исполнения.

1.2 Репрезентативность трасс

Для оценки репрезентативности трасс обычно сличаются целевые метрики сценариев исполнения и метрики, получаемые при анализе трасс, снятых с этих сценариев. Значения могут расходиться по разным причинам: трассировка может влиять на поведение приложения; используемая модель или методы анализа трасс могут искажать получаемые значения; на исследуемую нагрузку могут влиять внешние факторы.

Как уже было упомянуто ранее, данная работа сфокусирована на использовании трасс исполнения для анализа производительности разрабатываемых платформ. Поэтому основными целевыми метриками являются значения микроархитектурных счетчиков. Для оценки репрезентативности сличаются счетчики, снятые с трассируемого приложения, и счетчики, полученные от используемой модели (сконфигурированной для симуляции трассируемой системы).

При обработке некоторых записей трассы исполнения симулятором или другой моделью появляется необходимость компенсации состояния модели. Пример: из памяти прочитано новое значение, которое ранее не наблюдалось – требуется модификация памяти симулятора для корректного продолжения работы. Подобные изменения состояния модели производятся посредством исполнения дополнительного кода (далее – компенсационный код), что может существенно исказить собираемые метрики. Более того, компенсационный код фиксирован для каждой инструкции приложения – единожды произошедшая при симуляции инструкции трассы компенсация приводит к добавлению компенсационного кода для **всех** инструкций трассы, отвечающих той же инструкции приложения. Поэтому репрезента-

тивность трасс исполнения существенно зависит от компенсаций, требуемых при обработке.

Подробно методология сбора микроархитектурных метрик и оценки репрезентативности трасс в данной работе рассмотрена не будет. Однако будут приведены метрики, используемые для предварительной оценки качества получаемых трасс.

1.3 Виртуальное адресное пространство

Большинство современных компьютерных систем поддерживает виртуализацию адресного пространства [6]. Данная технология позволяет отображать (транслировать) адреса оперативной памяти, используемые кодом (виртуальные адреса) в адреса аппаратуры (физические адреса). Такой подход позволяет запускать множество приложений изолированно в одинаковой среде исполнения. Виртуальные адреса приложений могут пересекаться, однако при обращении к памяти будут использованы разные физические адреса. Создание карты трансляции адресов осуществляет операционная система по запросам приложений и загрузчиков.

При этом трансляция адресов обычно происходит постранично: виртуальное адресное пространство разбивается на непрерывные блоки одинакового размера (виртуальные страницы), последовательные адреса внутри одного блока отображаются в последовательные физические адреса (адреса физической страницы). Размер страницы - очень важный параметр системы, напрямую влияющий на производительность. Уменьшение размера страницы приводит к замедлению трансляции адресов, а увеличение - к большему расходу памяти. Типовые размеры страниц для современных систем: 4096, 16384 и 65536 байт.

К другим настройкам адресации можно отнести расположение запрещенных областей виртуального адресного пространства - областей, использование которых пользовательским кодом запрещено или ограничено. Такие области могут быть заняты данными операционной системы, зарезервированы для будущих расширений, запрещены для детектирования ошибок адресации (null page). Подобные области могут быть доступны из пользовательского кода только на чтение или недоступны вовсе.

Также многие современные системы имеют ограничения на создание отображений с правами на исполнение и запись (WX права) для повышения безопасности [30; 52].

1.4 Цель работы

Трассы исполнения могут быть использованы для анализа производительности разрабатываемых платформ. В случае, когда настройки системы памяти новой платформы отличаются от настроек трассируемого устройства, появляется необходимость адаптации трасс под конфигурацию новой системы. Это мотивирует разработку решения, позволяющего модифицировать записанные в трассах образы памяти в соответствии с ограничениями новой конфигурации. Для достижения данной цели достаточно разработать решение, позволяющее перемещать регионы образов памяти в соответствии с новыми ограничениями.

Цель работы: Разработать решение, позволяющее изменять трассы исполнения для модификации записанных в них образов памяти приложений в соответствии с ограничениями новой конфигурации системы памяти.

Задачи:

- Разработать решение, позволяющее перемещать регионы записанных в трассах образов памяти для удовлетворения ограничений новой конфигурации системы памяти.
- Протестировать решение на наборе целевых трасс заказчика.
- Оценить влияние производимых модификаций на репрезентативность трасс.

Работа будет осуществляться в рамках проекта для сбора и анализа трасс исполнения. Будут использованы некоторые компоненты проекта, внешние по отношению к разрабатываемому решению: функциональный симулятор целевой архитектуры, библиотека для чтения и записи трасс исполнения.

2 Обзор существующих решений

В данной главе будут рассмотрены существующие решения в области переноса кода и данных.

2.1 Позиционно-независимый код

Позиционно-независимый код (Position-Independent Code, PIC) [8; 31] - это код, который может быть размещен по любому адресу. PIC библиотека или приложение (модуль) напрямую не использует фиксированные адреса и для доступа к коду и данным использует относительные смещения или глобальную таблицу смещений (Global Offset Table, GOT) и таблицу связывания процедур (Procedure Linkage Table, PLT) [8; 49]. PIC модули могут быть загружены по любым адресам, что позволяет использовать их с технологией рандомизации адресного пространства (Address Space Layout Randomization, ASLR) и существенно упрощает динамическое связывание библиотек.

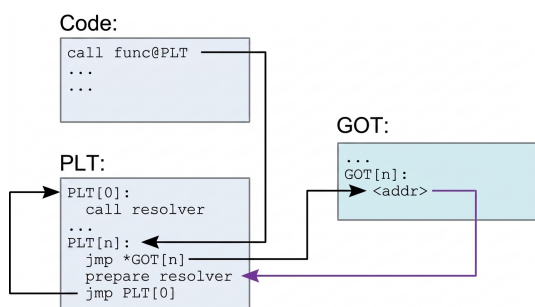


Рис. 3: Схема работы PLT: первый вызов процедуры [8].

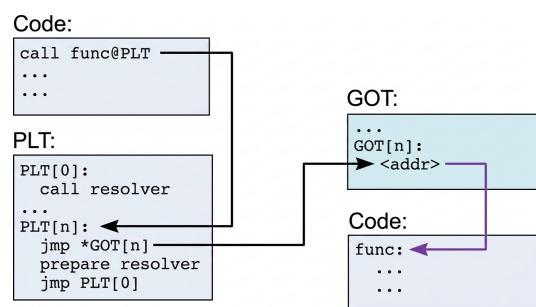


Рис. 4: Схема работы PLT: последующие вызовы [8].

Данная технология используется на целевой платформе, что упрощает модификацию затрассированного образа памяти, так как при переносе PIC сегмента целиком, относительная адресация внутри сегмента будет работать как в оригинальном сценарии (смещения между символами сегмента не изменятся). Однако исходные адреса кода и данных могут остаться на стеке, в куче и GOT. Так же относительная адресация внутри одного модуля перестанет корректно работать в случае переноса сегментов модуля с разными смещениями.

2.2 Компоновщики

В ходе статической компоновки [31] из набора объектных файлов и статических библиотек создается один готовый исполняемый файл или библиотека (статическая или динамическая). Сегменты кода и данных входных файлов объединяются, происходит разрешение

символов, с помощью данных из таблиц перемещений производятся замены адресов и смещений в коде и данных на финальные. Результирующий модуль содержит код и данные входных объектных файлов и статических библиотек, все символы разрешаются при компоновке.

Статическая компоновка позволяет осуществлять интер-модульные оптимизации (post-link optimizations) и уменьшает вычислительные расходы на загрузку приложения так как связывание происходит один раз при компоновке исполняемого файла. При этом данный метод может существенно увеличивать нагрузку на подсистему памяти во время исполнения множества приложений с одинаковыми зависимостями, поскольку код и данные каждой статической библиотеки будут продублированы в каждом приложении.

При динамической компоновке [31; 49] разрешение символов динамических библиотек откладывается до их загрузки (или даже непосредственно до использования). Компоновщик проверяет, что все внешние символы экспортируются какой-то из динамических библиотек, записывает имена динамических библиотек и их символов, создает GOT и PLT. Адреса динамически связываемых символов разрешаются во время их загрузки или использования специально сгенерированным кодом с помощью таблиц перемещений.

Динамическая компоновка позволяет ОС единожды загрузить динамическую библиотеку и предоставить ее код и данные в общее пользование всем зависимым процессам. Применение данного подхода позволяет существенно снижать потребление памяти современных систем. При этом динамическое связывание происходит непосредственно во время исполнения приложения, что приводит к дополнительным вычислительным расходам.

Следует отметить, что в обоих сценариях используются подготовленные компилятором и компоновщиком таблицы перемещений, т.е. локации заменяемых адресов и смещений известны.

2.3 Компиляторы

При однопроходной или двухпроходной компиляции адреса целевых меток кода могут быть неизвестны на момент генерации инструкций (здесь и далее в этом разделе имеются ввиду машинные инструкции) перехода на эти метки. Часто для корректного формирования инструкций перехода используется метод отложенного связывания (backpatching) [16]. При таком подходе инструкции перехода, для которых неизвестны смещения до целевых меток, генерируются с незаполненными смещениями и локации таких инструкций помещаются в список для дальнейшего исправления. Когда расположение целевой метки становится известным (например после обработки всей единицы трансляции), смещения зависимых инструкций заполняются корректными значениями.

Данный подход позволяет генерировать код без дополнительных полных проходов по промежуточному представлению. В качестве недостатков этого метода можно выделить затраты памяти на хранение вспомогательной информации и потенциальное усложнение оптимизаций, переставляющих участки кода.

2.4 Сборщики мусора ВМ

Среды выполнения многих языковых виртуальных машин (ВМ) включают сборщики мусора [25; 47], которые помимо освобождения неиспользуемой памяти могут перемещать используемые данные для решения проблемы фрагментации памяти, улучшения кеш-локальности и других оптимизаций.

Одним из наиболее распространенных семейств алгоритмов сборки мусора является семейство трассирующих сборщиков мусора. Зависимости между объектами, расположенными в памяти виртуальной машины, можно представить в виде направленного графа (граф объектов). Вершины графа - объекты, ребра - ссылки между объектами. Для определения того, какие объекты можно удалять и как обновлять ссылки на еще используемые объекты при их перемещении, осуществляются обходы графа из "корневых" объектов: объектов расположенных на регистрах и стеке ВМ, глобальных объектов, и т. д. Объекты, недостижимые из корневых объектов, можно очищать. А для перемещаемых объектов в ходе такого обхода собираются все ссылки, которые необходимо обновить.

Описанная методология позволяет эффективно освобождать память (в том числе очищать объекты, использующие циклические ссылки) и осуществлять дефрагментацию используемой памяти.

2.5 Динамическое восстановление вырезанных таблиц перемещений

Старые бинарные выпуски библиотек могут являться позиционно зависимыми, что делает системы, которые используют подобные версии библиотек уязвимыми для атак повторного использования кода (code reuse attacks). В [38] представлен метод динамического восстановления таблиц перемещений для позиционно зависимых библиотек, из которых была вырезана данная информация. Технология применяется для частичной рандомизации адресов загрузки позиционно зависимых библиотек, что повышает безопасность уязвимых систем.

При загрузке позиционно зависимой библиотеки код загружается по случайным адресам, а адреса, зарезервированные для кода библиотеки (старые адреса кода), запрещаются для использования через механизмы операционной системы, такие как mmap. При досту-

пе к старым адресам кода производится обработка системного исключения, в ходе которой анализируется паттерн обращения и либо осуществляется замена адреса на перемещенный, либо обращение помечается как потенциально вредоносное и выводится ошибка.

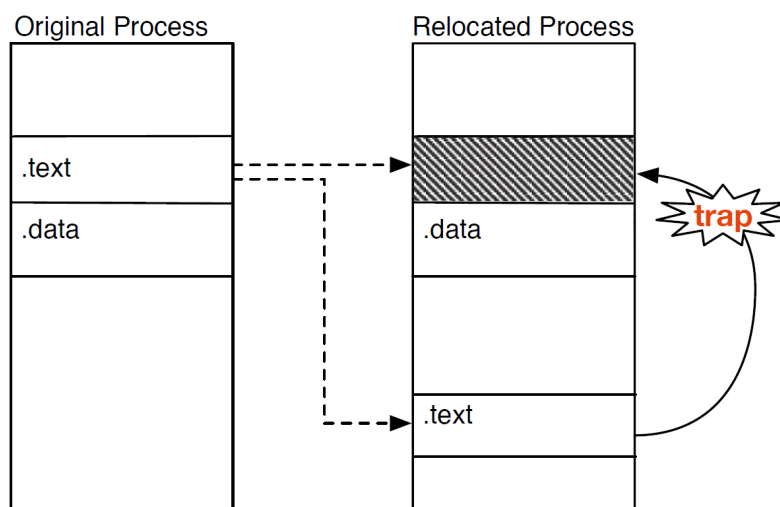


Рис. 5: Схема загрузки позиционно зависимых библиотек с частичной рандомизацией адресов [38]

2.6 Недостатки существующих решений

Все описанные в данной главе решения для переноса кода и данных либо используют подготовленные заранее локации адресов и смещений, либо восстанавливают эти локации из доступной информации о коде, либо работают с ограниченным набором краевых случаев (например, перемещают только код).

Локации адресов и смещений, находящихся среди данных трасс исполнения, известны только частично: записи для обращений в память содержат адреса - но предшествующий поток данных для этих адресов неизвестен; известны адреса исполняемого кода - но локации, по которым хранятся адреса и смещения для переходов потока управления отсутствуют. При этом необходима поддержка переноса различных областей памяти - кучи, стека, кода, константных данных, и т. д.

Недостатки существующих подходов в рамках ограничений рассматриваемой задачи мотивируют разработку нового решения, восстанавливающего информацию о потоках исполнения и данных для детектирования адресов и смещений в трассе.

3 Решение задачи

В данной главе будет рассмотрено решение поставленной задачи. Решение разработано на языке программирования C++.

3.1 Архитектура решения

Общая схема решения представлена на рис. 6.

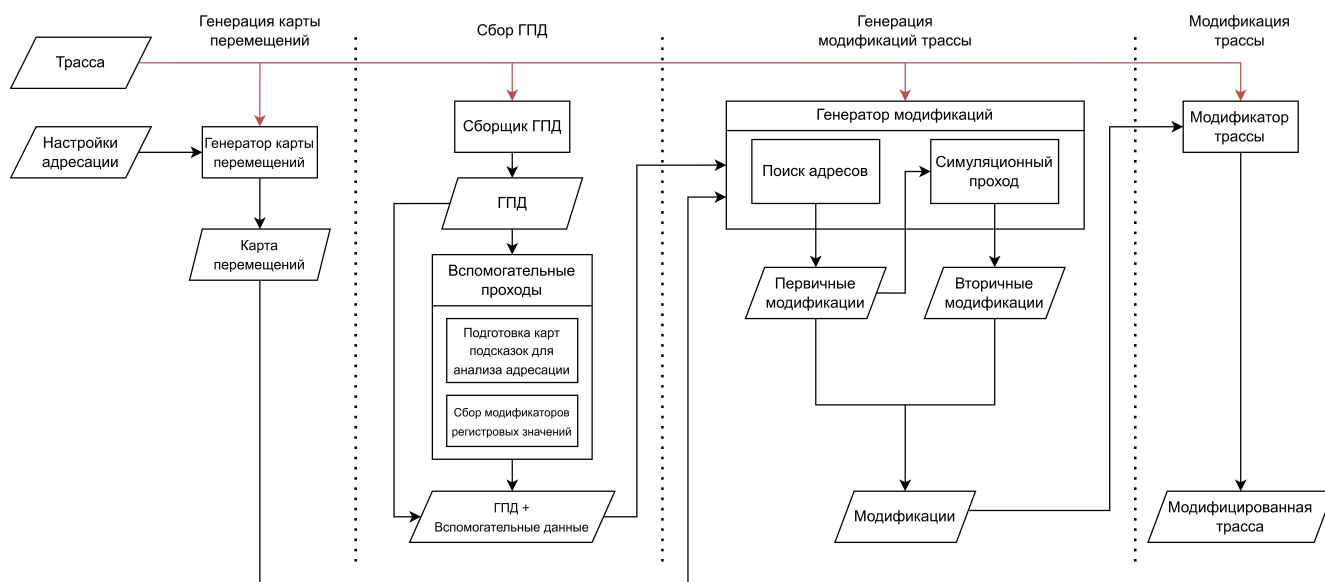


Рис. 6: Общая схема решения.

Обработка трассы происходит в четыре стадии:

1. Генерация карты перемещений

На этой стадии генерируется карта перемещений областей памяти, удовлетворяющая новым ограничениям, обусловленным изменением настроек адресации.

2. Сбор графа потока данных (ГПД)

На данной стадии осуществляется симуляционный проход по трассе, в ходе которого собирается ГПД. Затем осуществляется ряд обходов графа, в ходе которых собирается вспомогательная информация, размечающая его.

3. Генерация модификаций трассы

Генерация происходит в два этапа:

(а) Поиск адресов на ГПД и генерация первичных модификаций

- (b) Симуляционный проход по модифицированной трассе и генерация вторичных модификаций

4. Запись модифицированной трассы

Генерируемые модификации могут изменять существующие и добавлять новые записи, обеспечивая консистентность выходной трассы. Примеры возможных модификаций были представлены в разделе 1.1. Некоторые модификации могут снижать репрезентативность выходной трассы, так как будут приводить к изменению целевых счетчиков при дальнейшей ее обработке.

3.2 Генерация карты перемещений

На данной стадии с трассы собирается карта прав доступа для всех используемых приложением адресов. Далее производится слияние близких областей памяти с разрешенными комбинациями прав доступа. Максимальное расстояние для слияния областей является параметром командной строки. Затем производится перемещение полученных областей памяти в соответствии с новыми ограничениями адресации:

- При наличии в исходной карте прав доступа областей с запрещенными комбинациями прав, выводится ошибка - корректное преобразование трассы невозможно.
- Области, при слиянии прав доступа которых, получится комбинация прав, запрещенная новыми настройками адресации, размещаются на расстоянии превышающем новый размер страниц памяти.
- Также перемещение производится только в области памяти разрешенные новой конфигурацией.

Полученная карта перемещений используется последующими стадиями для анализа адресации и корректной замены адресов. Пример генерации карты перемещений приведен на рис. 7.

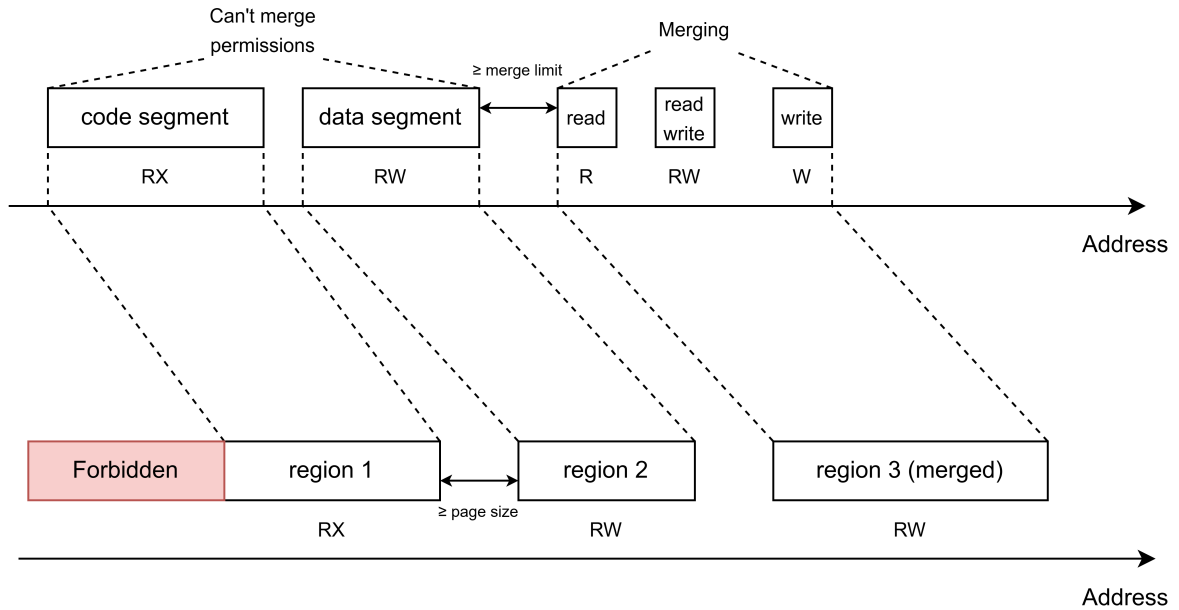


Рис. 7: Пример генерации карты перемещений.

3.3 Сбор графа потока данных

Обозначим $G = (V, E)$ - граф потока данных, где V - множество вершин графа, $E \subseteq V \times V$ - множество ребер графа. Так же обозначим: $\mathcal{P}(E)$ - множество всех подмножеств ребер графа. Вершины графа моделируют инструкции трассы. Ребра - зависимости между инструкциями по данным. С каждым ребром ассоциируются значение и локация, по которой данное значение располагалось во время трассировки, например регистр или область памяти. Термины вершина, динамическая инструкция и, если это не приводит к неоднозначности, инструкция используются взаимозаменяемо. Для представления графа потока данных используется реализация списка смежности из библиотеки boost graph (`boost::adjacency_list`) [10; 11].

Boost graph позволяет ассоциировать с вершинами и ребрами пользовательские данные. Для инструкций хранятся: номер инструкции в трассе, адрес инструкции, ее код. Для ребер хранятся: восьми-байтное значение и его локация (уже упомянутые ранее) - в памяти или на регистре общего назначения, модификаторы значения: тип расширения, размеры битовых сдвигов, и т. п. Заметим, что размер значения фиксированный, зависимости по значениям большего размера отображаются с помощью нескольких ребер.

На графе есть вершины, не соответствующие реальным инструкциям (вершина "начальное состояние" на рис. 8). Будем называть такие вершины псевдо-инструкциями. Такие вершины всегда являются истоками и используются для отображения зависимостей, не порожденных отдельными инструкциями, или для добавления вспомогательных зависимостей:

- Вершина начального состояния - для зависимостей от начального состояния памяти или начальных значений регистров.
- Вершина внешних зависимостей по памяти - для зависимостей по памяти, модифицированной другим потоком или ОС.

Пример: Из памяти прочитано значение, не соответствующее текущему образу памяти. Данное значение могло быть записано другим потоком или ОС во время системного вызова или обработки системного исключения. На графе данная зависимость будет представлена в виде ребра, выходящего из вершины внешних зависимостей и входящего в инструкцию, прочитавшую новое значение.

- Вершина сложных зависимостей по памяти - для зависимостей по памяти от нескольких инструкций.

В данный момент полноценная поддержка таких зависимостей отсутствует.



Рис. 8: Пример графа потока данных.

Сбор графа потока данных начинается с симуляционного прохода по трассе (блок схема алгоритма представлена на рис. 9), в ходе которого собираются:

- Граф потока данных.
- Данные вершин.
- Данные ребер, кроме модификаторов значений (таких как размеры битовых сдвигов и т. п.).
- Карта, отображающая идентификаторы инструкций в дескрипторы, с помощью которых осуществляется доступ к вершинам.

- Карта зависимостей инструкций от флагов условий.

Заметим, что зависимости между инструкциями по флагам условий хранятся отдельно от графа потока данных. Данное решение позволяет разделить разнородные зависимости, что существенно упрощает обработку.

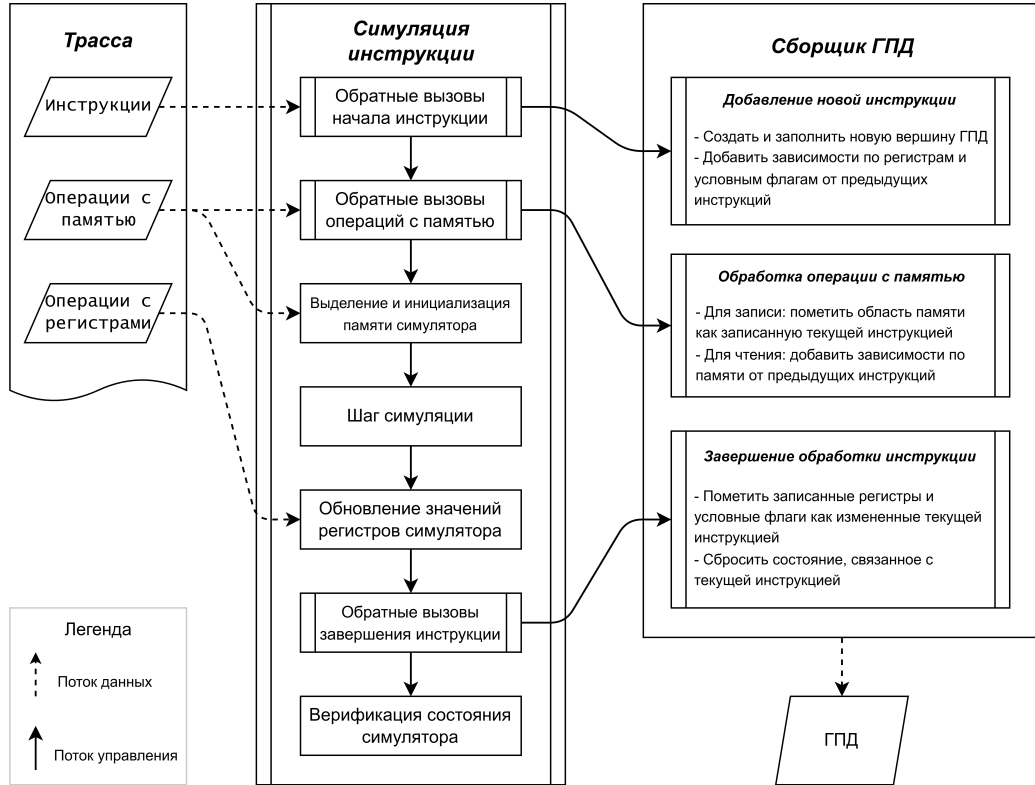


Рис. 9: Блок-схема алгоритма симуляционного прохода по трассе.

После симуляционного прохода производится ряд обходов графа, в ходе которых собираются:

- Карты подсказок для алгоритмов анализа адресов.
- Модификаторы значений для ребер графа.

Карты подсказок позволяют получать вспомогательные данные для вершин и ребер графа (примеры фрагментов ГПД и сформированных для них подсказок представлены на рис. 10):

- $\text{src_edge_hints} : E \rightarrow E$ задает порядок обратного обхода графа для алгоритма поиска адресов в ряде краевых случаев. Формируется с помощью набора эвристик.
- $\text{dst_edge_hints} : E \rightarrow 2^E = \text{src_edge_hints}^{-1}$ задает порядок прямого обхода графа для алгоритма поиска адресов в ряде краевых случаев. Задает обратное многозначное отображение для src_edge_hints .

- $\text{base_edge_hints} : V \rightarrow E$ отображает вершины в соответствующие им входные ребра базовых адресов.
- $\text{offset_edge_hints} : V \rightarrow E$ отображает вершины в соответствующие им входные ребра смещений адресов.

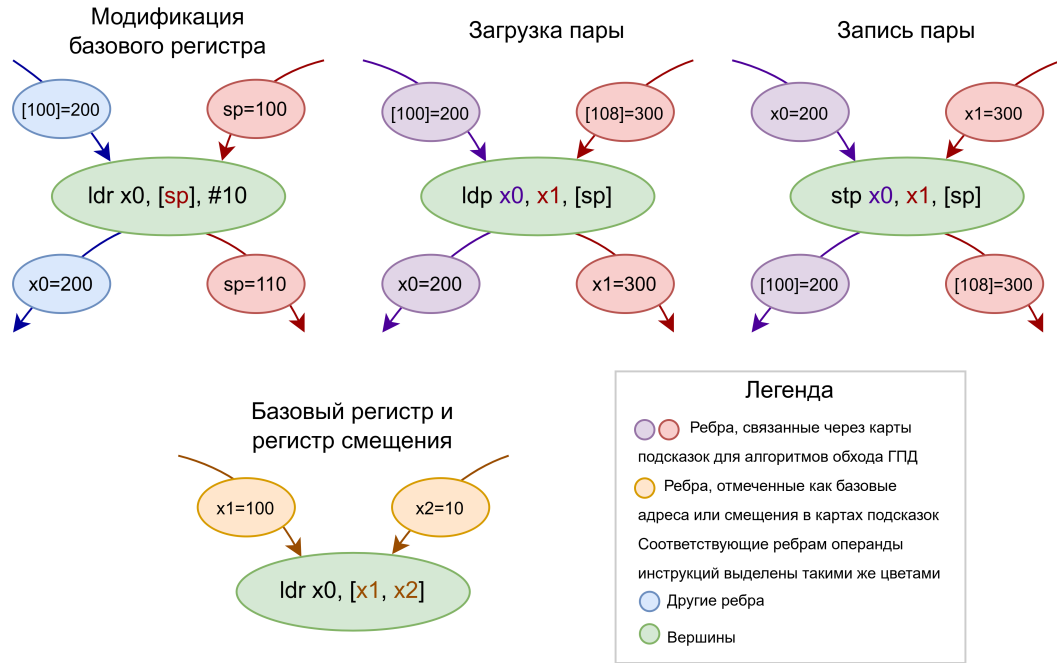


Рис. 10: Примеры фрагментов ГПД и сформированных для них подсказок.

На рис. 11 представлена UML диаграмма [34; 46] модуля, отвечающего за сбор и анализ графа потока данных (модуль ГПД).

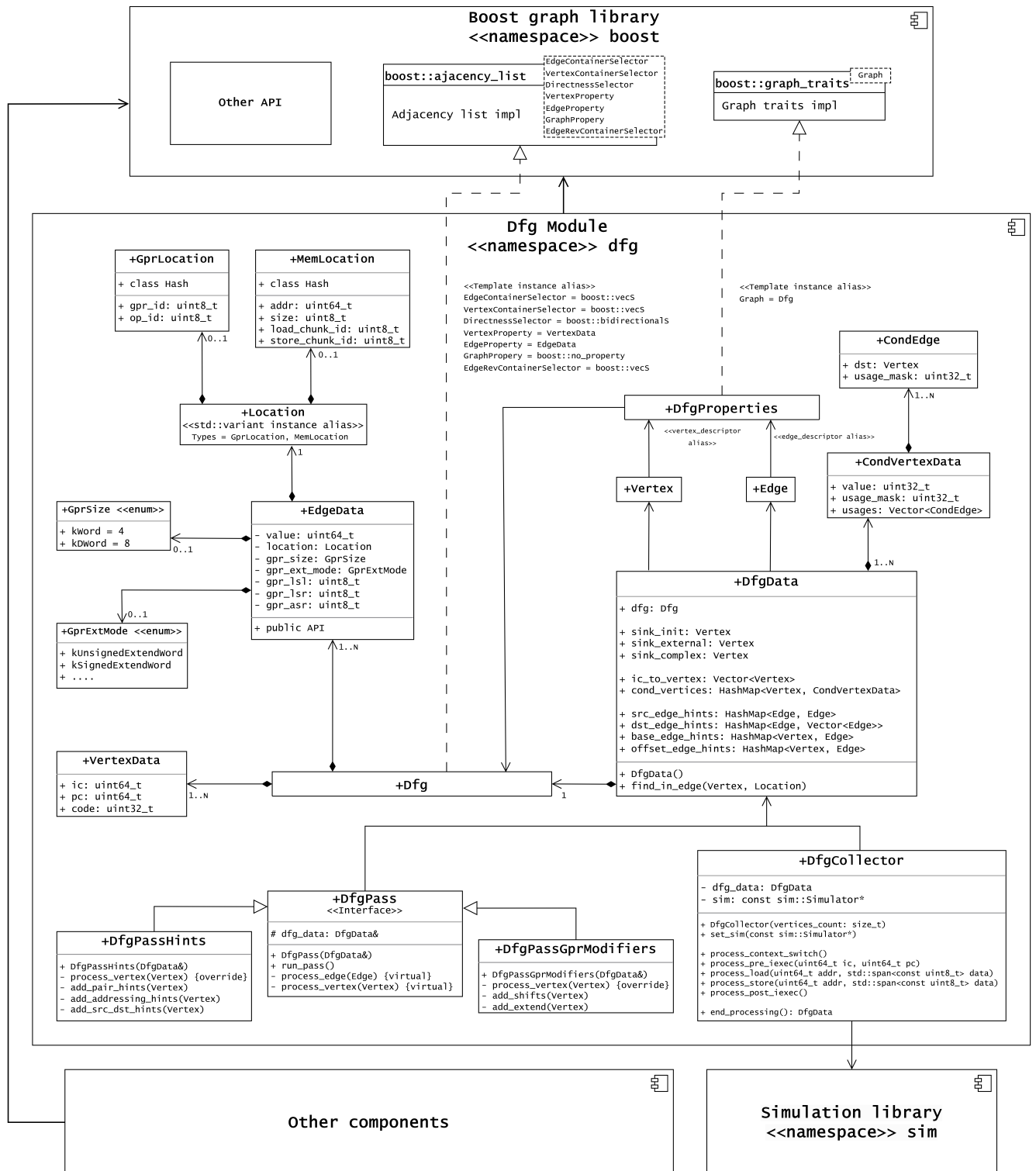


Рис. 11: UML диаграмма модуля ГПД.

3.4 Генерация модификаций трассы

Для генерации трасс с консистентными потоками управления и данных, необходимо поддержать модификации для следующих данных трассы (см. раздел 1.1):

- Начальное регистровое состояние

Начально регистровое состояние может содержать адреса, которые необходимо заменить в соответствии с картой перемещений.

- Обращения к памяти

Обращения к памяти должны происходить по измененным адресам. Также, данные этих обращений могут сами по себе быть адресами.

- Значения регистров общего назначения

Регистровые значения, используемые в качестве адресов, должны быть заменены.

Для упрощения задачи запретим добавлять записи типа "обращение в память" и модифицировать размеры и типы обращений для существующих записей: поток данных выходной трассы должен быть похож на исходный, но с замененными адресами. Таким образом, будут использованы четыре вида модификаций для записей трассы (три из которых генерируются на рассматриваемой стадии обработки):

- Модификации начального регистрового состояния

- Модификации чтений из памяти

- Данные могут быть заменены, если из памяти читается значение, используемое в дальнейшем в качестве адреса.
- Адрес чтения должен быть получен из исходного с помощью карты перемещений.
- Новые записи не добавляются.

- Модификации записей в память

- Данные получаются из симуляции: в память может записываться перемещенный адрес (возможно, преобразованный) или старые данные.
- Адрес записи должен быть получен из исходного с помощью карты перемещений.
- Новые записи не добавляются.
- Не генерируются на рассматриваемой стадии обработки: преобразования тривиальные и осуществляются при записи выходной трассы.

- Модификации записей в регистры общего назначения

- Регистровые значения для существующих записей могут быть изменены.

- Могут быть добавлены новые записи, исправляющие потоки управления и данных.

Генерация модификаций происходит в два этапа:

1. Поиск адресов и смещений на ГПД

На этом этапе производится поиск ребер, значения которых (возможно, после некоторых преобразований) используются в качестве адресов. Для найденных ребер генерируются первичные модификации трассы.

2. Симуляционный проход по модифицированной трассе

На этом этапе детектируются расхождения потока управления и потока данных симулятора с ожидаемым поведением на модифицированной трассе. При детектировании расхождений генерируются вторичные модификации трассы, исправляющие ошибки.

Предложенный алгоритм по построению поддерживает любые паттерны адресации в трассе, но может генерировать трассы с большим количеством компенсаций. Подробнее свойства предложенного алгоритма будут рассмотрены в дальнейшем.

3.4.1 Поиск адресов и смещений

Введем два маркера для ребер ГПД: красный маркер отмечает ребра, обработанные в ходе обратного обхода ГПД; зеленый маркер отмечает ребра, попавшие в очередь прямого обхода ГПД. В данную очередь будем помещать только ребра, еще не покрашенные зеленым маркером.

Будем называть два ребра близкими по значению если абсолютная разность их значений меньше некоторого порогового значения, являющегося гиперпараметром решения (обозначим как `ADDR_SIMILARITY_THRESHOLD`). Также будем называть ребро $e \in F \subseteq E$ ближайшим по значению к ребру g среди ребер F , если e и g близки по значению и пара (e, g) обладает минимальной абсолютной разностью значений среди всех пар $F \times \{g\}$.

Сначала с ГПД собираются ребра, значения которых используются целевыми инструкциями в качестве адресов или считаются адресами по другим косвенным признакам (адресные ребра). К таким значениям относятся: базовые адреса обращений в память; адреса переходов потока управления; адреса, используемые для операций с кешами; значения, которые сравниваются с адресами или вычитаются из них; и т.п.

Для каждого адресного ребра, строится обратный путь на ГПД, начинающийся с этого ребра. Построение выполняется последовательно. На каждом шаге для текущего конечного

ребра пути (обозначим это ребро как e) выбирается следующее ребро из множества рёбер, входящих в начальную вершину e . Построение пути осуществляется до выполнения одного из условий останова: достижения псевдо-инструкции; ошибки поиска следующего ребра; достижения ребра, окрашенного красным маркером; и т.п. Конечное ребро построенного пути будем называть исходным адресным ребром. Для таких ребер, а также ребер пути с локациями в памяти, генерируются первичные модификации трассы, что позволяет заменить большую часть используемых адресов (так как изменения будут распространяться по потоку данных) путем малого количества модификаций трассы. Все ребра построенного пути красятся красным маркером, а исходные адресные ребра также помещаются в очередь прямого обхода ГПД (если они еще не окрашены зеленым маркером). Блок схема алгоритма построения обратных путей на ГПД представлена на рис. 12.

Затем производится обработка ребер в очереди прямого обхода ГПД. Для каждого ребра e строится обратный путь (если он еще не был построен) и осуществляется поиск ребер для дальнейшего обхода. Найденные ребра добавляются в очередь прямого обхода:

- Ребра с такой же начальной вершиной и локацией
- Ребра, значения которых сравниваются со значением e

Если e входит в вершину v , являющуюся инструкцией сравнения, то все входные ребра v добавляются в очередь.

- Ребра, значения которых участвуют в вычитании со значением e

Сценарий аналогичен предыдущему, но значения добавляемых ребер должны быть больше некоторого порогового значения, являющегося гиперпараметром решения (обозначим как `ADDR_MIN_VALUE`).

- Ребра, соответствующие e в `dst_edge_hints`.
- Ребра, выходящие из v , близкие по значению к e

Данные ребра добавляются только если для e нет записей в `dst_edge_hints`.

Прямой обход ГПД позволяет найти дополнительные адресные ребра для построения обратных путей на графе и генерации первичных модификаций. Этот обход нужен, так как часть значений в трассе может не использоваться в качестве адресов напрямую, но может сравниваться с адресами, использоваться для подсчета хеш сумм адресов, и т. п. Часть таких значений будет найдена при данном обходе, что позволит в дальнейшем сделать меньше вторичных модификаций (которые всегда снижают репрезентативность трассы).

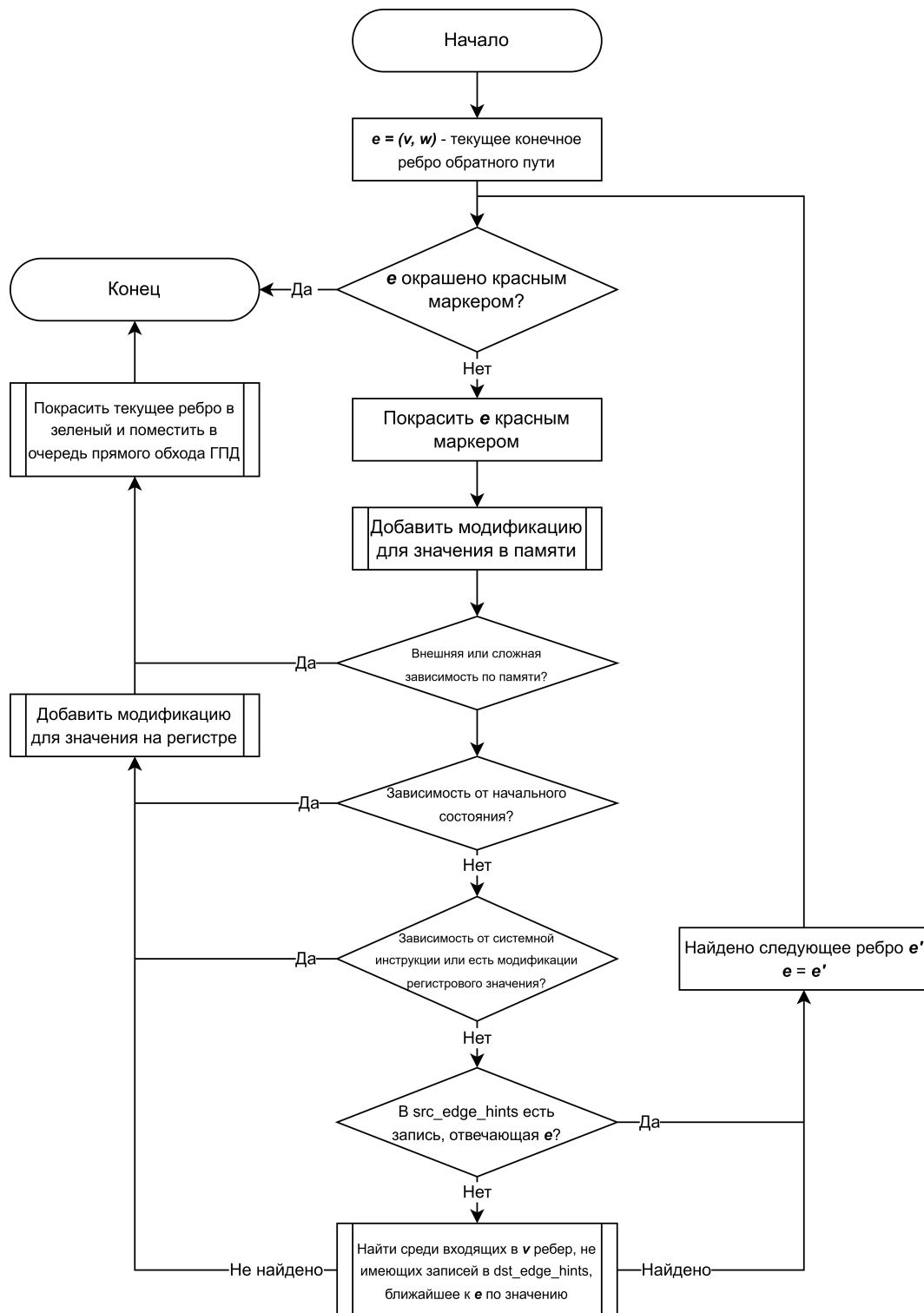


Рис. 12: Блок схема алгоритма построения обратных путей на ГПД.

3.4.2 Симуляционный проход по модифицированной трассе

На данной стадии производится проход по трассе, аналогичный изображенному на рис. 9. Но записи трассы преобразуются в соответствии со сгенерированными модификациями. Также до и после каждого симуляционного шага потоки управления и данных симулятора проверяются на расхождение с ожидаемым поведением. При детектировании расхождения,

состояние симулятора исправляется и генерируются вторичные модификации трассы, соответствующие исправлению. В случае расхождения после симуляционного шага, состояние симулятора восстанавливается и шаг делается повторно (с примененными исправлениями). Описанный подход позволяет генерировать трассы с консистентными потоками управления и данных. Блок схема алгоритма представлена на рис. 13.

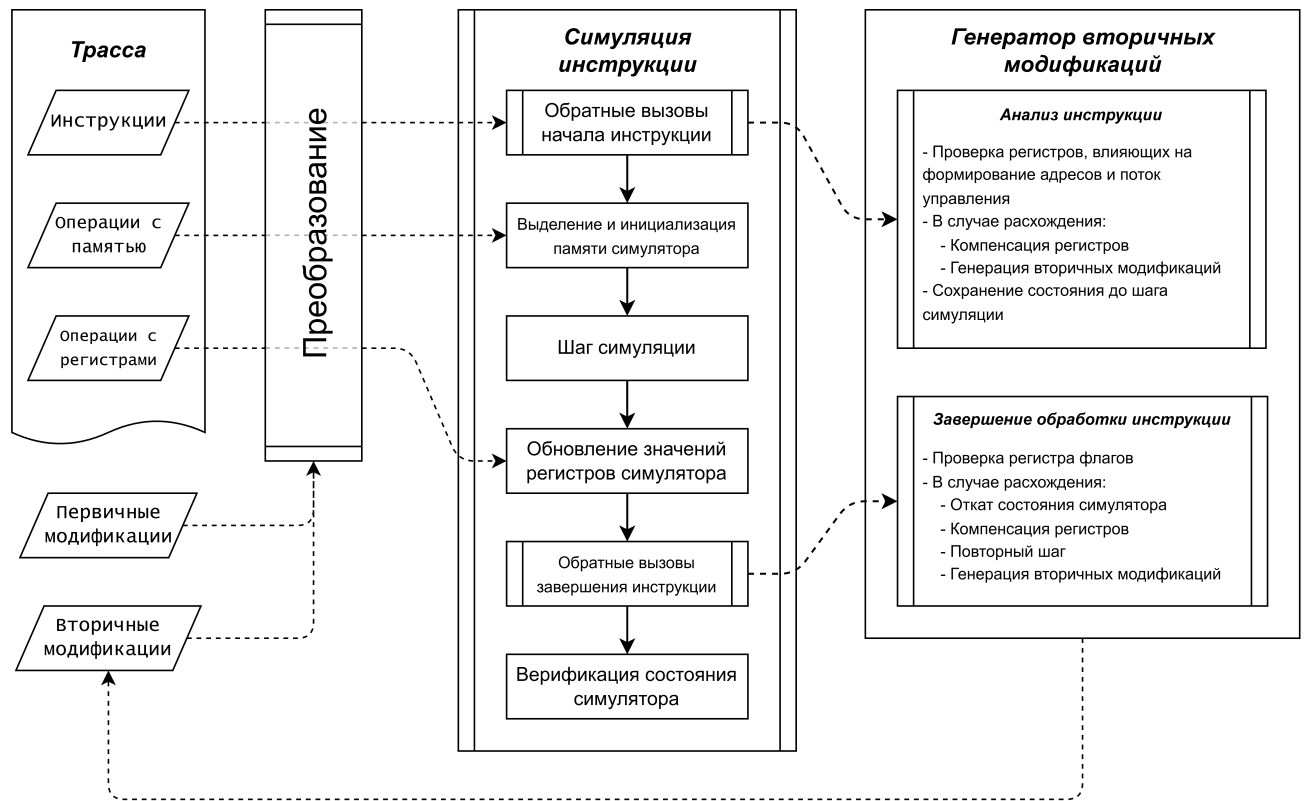


Рис. 13: Блок схема симуляционного прохода по модифицированной трассе.

3.5 Запись модифицированной трассы

На данной стадии обработки происходит проход по исходной трассе, в ходе которого оригинальные записи преобразуются с помощью подготовленных модификаций (первичных и вторичных) и записываются в выходную трассу. Так же во время обработки происходит симуляция по модифицированной трассе, аналогичная подходу описанному в разделе 3.4.2. Симуляция используется для верификации записываемой трассы и получения данных для некоторых записей.

3.6 Симуляция по трассе

Рассмотрим подробнее симуляцию по трассе, ранее упомянутую в разделах 1.1, 3.3, 3.4.2 и 3.5. В процессе симуляции данные из трассы пробрасываются в состояние симулятора, таким образом воспроизводится записанное в трассе исполнение и собираются необходимые

данные, отсутствующие в трассе (например значения регистров в определенных точках). Также обычно проверяются консистентность потока управления и данных.

Для ряда рассмотренных задач применяется симуляция по модифицированной трассе. Заметим, что модифицированная трасса при этом еще не записана в выходной файл, а конструируется налету из записей исходной трассы и подготовленных модификаций. Функционал обычной симуляции по трассе получается переиспользовать с помощью наследования и переопределения виртуальных функций. Переопределенные методы-обработчики подменяют исходные записи трассы (и/или добавляют новые), а после вызывают методы базового класса на полученных данных. Пример переопределенного метода для обработки модифицированной трассы приведен в листинге 1.

```
/// Trace-based simulation for memory record.
/// Called before related instruction simulation for data injection.
/* virtual */ void TraceSim::process_pre_mem(const trace::MemEvent &ev) {
    switch (ev.kind) {
        case trace::MemEventKind::Load:
            // Inject trace data
            m_sim->write_virt_mem(ev.addr, ev.data);
            break;
        case trace::MemEventKind::Store:
            // Allocate touched memory region
            m_sim->allocate_virt_mem(ev.addr, ev.data.size());
            break;
    }
}

/// Patched trace-based simulation for memory record.
/// Patch trace record and delegate to base method.
void PatchedTraceSim::process_pre_mem(const utl::MemEvent &ev) /* override */ {
    // Patch event
    auto new_ev = m_patches.patch_mem_event(m_current_instruction, ev);
    // Delegate
    TraceSim::process_pre_mem(new_ev);
}
```

Листинг 1: Переопределение метода-обработчика записей доступа в память

Также класс симулятора по трассе поддерживает добавление обратных вызовов (callback), что позволяет переиспользовать его для различных симуляционных задач решения. UML диаграмма симуляционного модуля представлена на рис. 14.

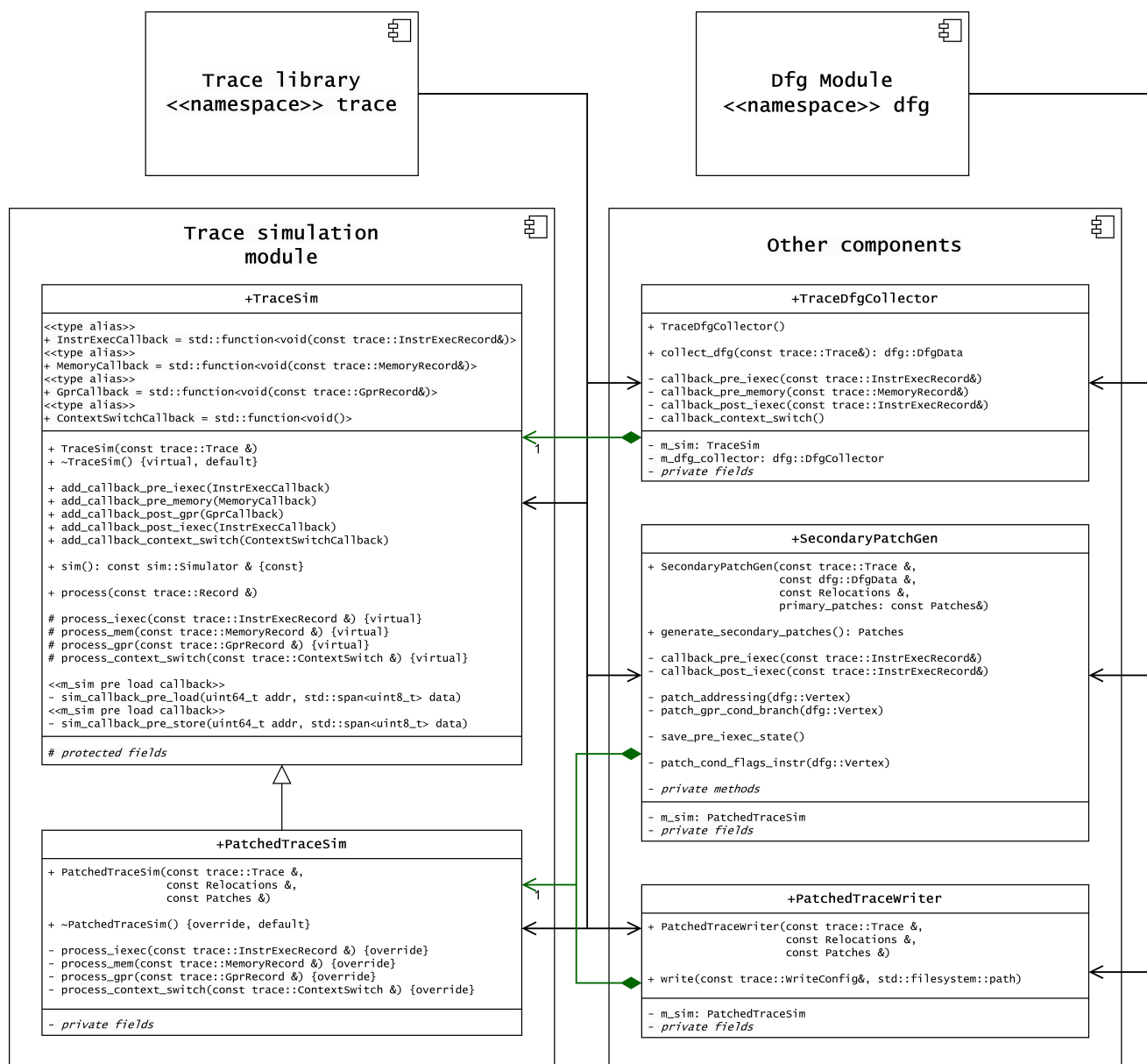


Рис. 14: UML диаграмма симуляционного модуля.

3.7 Тестирование

Для тестирования программного обеспечения (ПО) часто применяются следующие подходы:

- Юнит тестирование - проверяется корректность работы отдельных функций или методов изолированно от других компонентов [12; 13; 20].
- Модульное тестирование - проверяется работа отдельных модулей. Тестируются корректность взаимодействий внутри модуля и его интерфейсов.
- Сквозное тестирование (end-to-end testing) - решение тестируется полностью на целевых сценариях и данных.

Для юнит и модульного тестирования решения используется фреймворк GoogleTest [20] и тестовая инфраструктура используемой системы сборки CMake [23; 28]. Так же возможно добавление сквозных тестов с использованием реальных или синтетических трасс и инфраструктуры проекта.

Дизайн ключевых компонентов решения позволяет применять юнит и модульное тестирование, возможно после нескольких незначительных изменений. Основной задачей для поддержки подобного тестирования является отделение компонентов решения от библиотеки для чтения трасс. Это позволит осуществлять частичную обработку тестовых данных без наличия реальной трассы. Примером подобного отделения является публичный интерфейс модуля ГПД (см. рис. 11): класс *DfgCollector* принимает данные, не привязанные явно к библиотеке для чтения трасс (хотя формат данных сильно похож на формат записей трассы). Это позволяет собирать ГПД и вспомогательные данные на тестовых сценариях, а затем проверять консистентность, инварианты и выполнение тестовых условий. Для упрощения описания тестовых сценариев была реализована вспомогательная инфраструктура, в упрощенном виде представленная в листингах 2 и 3. Пример юнит теста, использующего описанную инфраструктуру представлен в листинге 4.

С помощью описанной тестовой инфраструктуры было добавлено 7 юнит тестов, покрывающих базовый функционал для работы с ГПД и ряд краевых случаев. Примененный подход показал свою эффективность: тестовые сценарии описываются легко и могут быть переиспользованы несколькими тестами для проверки различных компонентов. Полное тестирование модуля ГПД и других модулей может быть осуществлено в рамках дальнейшего развития решения.

```
/// DFG builder for testing
struct DfgBuilder {
    /// Set sim gpr
    void set_gpr(uint8_t id, uint64_t value);

    /// Simulator data getters
    uint64_t pc() const;
    uint64_t sp() const;
    uint64_t icount() const;

    /// Sink vertices getters
    dfg::Vertex init() const;
    dfg::Vertex extmod() const;
    dfg::Vertex complex() const;

    /// Info about test basic block
    struct BasicBlockInfo {
        const dfg::DfgData &dfg_data;
        /// Global icount range
        uint64_t start_ic = 0;
        uint64_t end_ic = 0;

        // Helper getters ...
    };

    /// Info about load
    struct Ld {
        uint64_t addr = 0;
        std::vector<uint8_t> data;
    };

    /// Info about store
    struct St {
        uint64_t addr = 0;
        std::vector<uint8_t> data;
    };

    /// Add specified basic block & return aggregated info about it
    template <class... Args>
    BasicBlockInfo bb(Args &&...args);

    /// Finalize DFG building
    void finalize();
};
```

Листинг 2: Класс для генерации тестовых ГПД.

```

/// Fixture for `DfgCollector` tests
struct DfgCollectTest : public DfgBuilder, public ::testing::Test {
protected:
    /// Check that vertex with specified icount exists and return descriptor for it
    dfg::Vertex expect_vertex(uint64_t ic);
    /// Check that specified edge exists and return descriptor for it
    dfg::Edge expect_edge(dfg::Vertex src, dfg::Vertex dst,
        uint64_t value, uint8_t gpr_id, uint8_t op_id);
    /// Check that specified edge exists and return descriptor for it
    dfg::Edge expect_edge(dfg::Vertex src, dfg::Vertex dst,
        uint64_t value, const MemoryLocation &loc);
    /// Check that edge `src` has a destination hint pointing to `dst` edge
    void expect_src_dst_hint(dfg::Edge src, dfg::Edge dst);
};

```

Листинг 3: Класс-фикстура для юнит тестирования класса DfgCollector.

```

/// Test for `ldp` instruction processing based on `DfgCollectTest` fixture
TEST_F(DfgCollectTest, ldpImmediateOffset) {
    auto bb1 = bb(
        "ldp x3, x4, [sp, #16]",
        Ld{sp() + 16, {uint64_t{10}, uint64_t{20}}},
        "add x5, x3, x4"
    );
    DfgBuilder::finalize();

    expect_edge(init(), bb1.vertex(0), sp(), uint8_t{31}, uint8_t{2});
    // Check edges from initial memory to `ldp`
    auto edge_init_to_ldp_mem0 = expect_edge(init(), bb1.vertex(0), 10, ....);
    auto edge_init_to_ldp_mem1 = expect_edge(init(), bb1.vertex(0), 20, ....);
    // Check edges from `ldp` to `add`
    auto edge_ldp_to_add_x3 = expect_edge(bb1.vertex(0), bb1.vertex(1), 10, ....);
    auto edge_ldp_to_add_x4 = expect_edge(bb1.vertex(0), bb1.vertex(1), 20, ....);

    // Check hints for related edges
    expect_src_dst_hint(edge_init_to_ldp_mem0, edge_ldp_to_add_x3);
    expect_src_dst_hint(edge_init_to_ldp_mem1, edge_ldp_to_add_x4);
}

```

Листинг 4: Пример юнит теста для класса DfgCollector.

3.8 Оценка решения

В данном разделе будет произведена качественная и количественная оценка решения.

3.8.1 Пространственная и временная сложность

Рассмотрим алгоритмическую сложность [24] решения. Размеры трассы и количество итераций при проходе по ней пропорциональны количеству инструкций в трассе $N = |V|$.

Рассмотри временную и пространственную сложность стадий обработки трассы:

- Генерация карты перемещений

Временная сложность - $O(N)$ (полный проход по трассе). Пространственная сложность - $O(N)$ (агрегация данных с записей трассы).

- Сбор ГПД

Временная сложность - $O(N)$ (полный проход по трассе). Пространственная сложность - $O(N + K)$. Где $K = |E|$ - количество ребер ГПД.

- Генерация модификаций трассы

На данной стадии производится два обхода ГПД: построение обратных путей и прямой обход. При этом ни одно ребро не обходится дважды, так как обработанные ребра отмечаются маркерами. Временная сложность обходов графа - $O(N + K)$. Также в конце производится симуляционный проход по модифицированной трассе, временная сложность - $O(N)$.

Результирующая временная сложность - $O(N + K)$

На данной стадии генерируются модификации трассы, количество которых пропорционально количеству инструкций и количеству ребер ГПД. Пространственная сложность - $O(N + K)$.

- Модификация трассы

Временная сложность - $O(N)$ (полный проход по трассе). Пространственная сложность - $O(1)$.

Заметим, что количество ребер, выходящих из одной инструкции ограничено константой (ограничение системы команд платформы). Значит выполняется асимптотическая оценка: $K = O(N)$. Таким образом, пространственная и временная сложность обработки трассы $O(N + K) = O(N)$.

3.8.2 Оценка репрезентативности трасс

Как было описано в разделе 1.2, компенсируемые записи могут существенно ухудшать репрезентативность трасс. При этом единожды произошедшая при симуляции инструкции трассы компенсация приводит к добавлению компенсационного кода для **всех** инструкций трассы, отвечающих той же инструкции приложения. Рассмотрим различные модификации трассы и их влияние на репрезентативность:

- Замена адреса для операции с памятью - не влияет на репрезентативность, так как не требует компенсации при дальнейшей обработке выходной трассы.
- Замена значения для записи в память - не требует компенсации (поток данных консистентен).
- Замена значения для чтения из памяти - может влиять на репрезентативность в некоторых сценариях.

При чтении значения, не соответствующего текущему образу памяти рассматриваемого потока, может потребоваться компенсация, аналогичная компенсации для внешней модификации памяти. Заметим, что если в оригинальной трассе уже присутствовала внешняя модификация памяти, репрезентативность существенно не изменится.

- Замена значения регистра в начальном состоянии - не требует дополнительной компенсации.
- Замена значения для записи в регистр общего назначения - не требует дополнительной компенсации.
- Добавление записи в регистр общего назначения - требует компенсации, будет снижать репрезентативность.

Для оценки влияния модификаций на репрезентативность трасс будем использовать кумулятивную метрику N_{comp} – количество инструкций трассы, при симуляции которых будет исполнен компенсационный код, появившийся из-за произведенных над трассой модификаций. Обозначим также $P_{comp} = \frac{N_{comp}}{N}$ - доля инструкций трассы, для которых будет добавлен компенсационный код. Данные метрики позволяют предварительно (без полноценного анализа выходных трасс на симуляторе) оценить качество произведенных модификаций и репрезентативность получаемых трасс.

3.8.3 Профилировка решения по памяти

Для получения грубой оценки потребления памяти различными компонентами использовалась утилита `heaptrack` [26], позволяющая отследить объемы выделений памяти различными участками кода. Данные `heaptrack` для набора тестовых трасс показывают, что в коде библиотеки `boost graph` выделяется больше 90 процентов используемой памяти. Значит большую часть потребляемой памяти занимает ГПД. При этом выполняется: $M_{\text{ГПД}} = O(N + K) = O(N)$, где $M_{\text{ГПД}}$ - объем памяти, занимаемый графом.

3.8.4 Тестовые запуски на целевых трассах

В данном разделе будут представлены результаты тестовых запусков на целевых трассах заказчика. Трассы сгруппированы по целевым сценариям исполнения, для каждого сценария приведено до 10 трасс. Для тестов используется одна из исследуемых конфигураций системы памяти: с увеличенным размером страниц и резервированием дополнительных областей под данные ядра ОС.

На этапе обработки трасс собирались следующие метрики:

- N - общее количество инструкций трассы.
- N_{comp} - количество инструкций, для которых будет добавлен компенсационный код.
- $P_{\text{comp}} = \frac{N_{\text{comp}}}{N}$ - доля инструкций, для которых будет добавлен компенсационный код. Будем считать трассы с $P_{\text{comp}} \leq 0.01$ репрезентативными.
- RSS_{max} - максимальный объем резидентной памяти (resident set size) [19; 32], занятый при модификации трассы.
- $P_{RSS} = \frac{RSS_{\text{max}}}{N}$ - объем резидентной памяти на инструкцию трассы. Данная метрика интересна так как объем занимаемой памяти линеен по количеству инструкций в трассе.
- T - время обработки трассы.
- $V = \frac{N}{T}$ - средняя скорость обработки (инструкций трассы в секунду).

После завершения обработки результирующие трассы были верифицированы с помощью функционального симулятора.

Общее количество тестовых трасс: $M = 272$

Трасс, прошедших обработку без ошибок: **264 (97%)**

Трасс, прошедших обработку и верификацию без ошибок: **217 (80%)**

Трасс, прошедших обработку и верификацию без ошибок с $P_{comp} \leq 0.01$: **132 (49%)**

Значения для P_{comp} представлены на рис. 15 - 17.

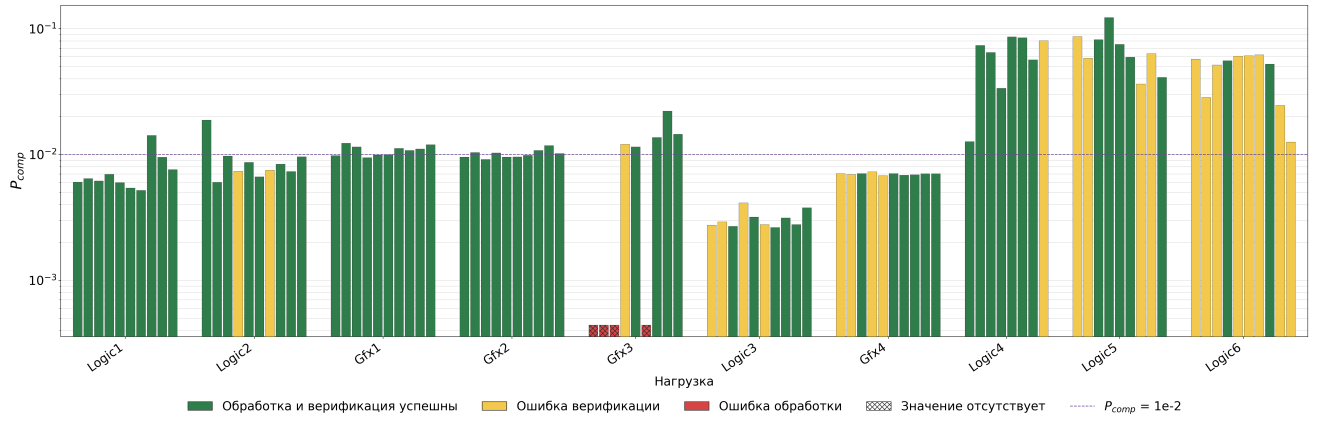


Рис. 15: P_{comp} на наборе тестовых трасс (логарифмический масштаб, 1/3)

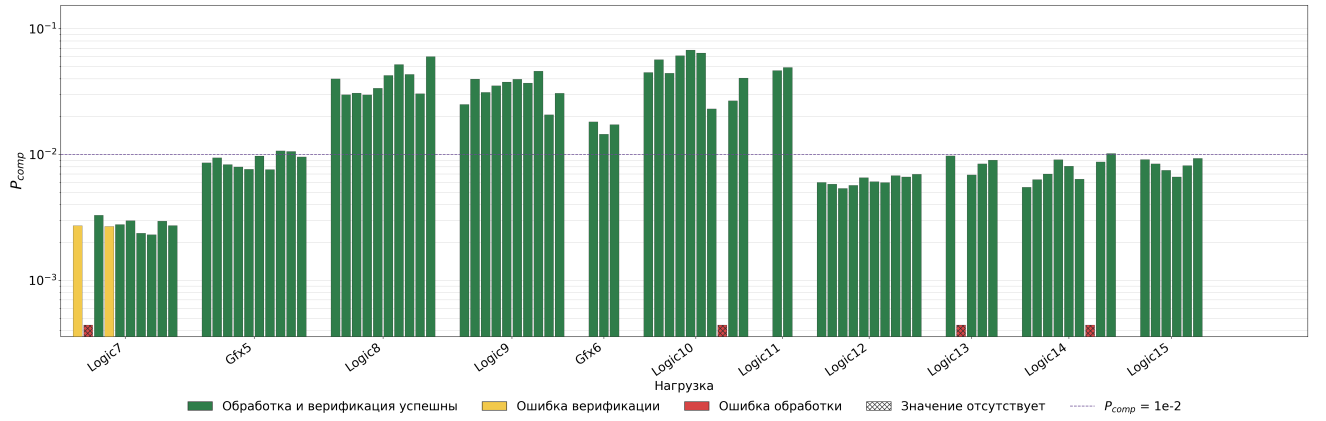


Рис. 16: P_{comp} на наборе тестовых трасс (логарифмический масштаб, 2/3)

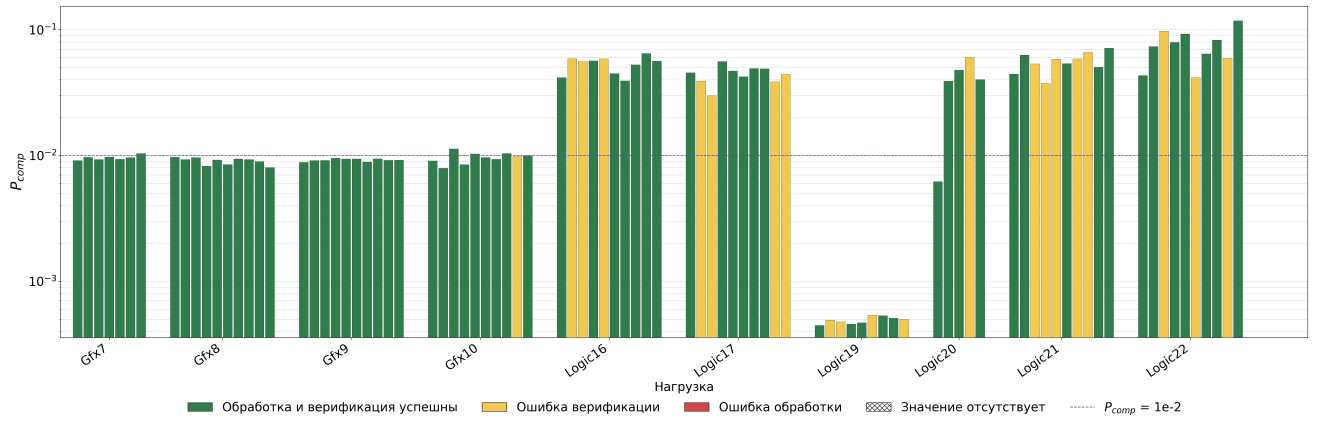


Рис. 17: P_{comp} на наборе тестовых трасс (логарифмический масштаб, 3/3)

Рассчитаем обобщающие показатели для P_{comp} , P_{RSS} и V .

Для P_{comp} приведено среднее арифметическое - доля компенсированных инструкций трассы характеризует качество выходной трассы, подобное усреднение позволит оценить долю компенсаций в случайно выбранной трассе. При этом не учитываются размеры тестовых трасс, так как на долю компенсаций в первую очередь влияют особенности потоков данных и управления. Также приведена медиана, так как на ряде тестовых трасс данная метрика существенно выше, что может говорить о наличии большого количества еще не поддерживаемых краевых случаев.

$$\overline{P_{comp}} = \frac{1}{M} \sum_i P_{comp_i} = (2.50 \pm 0.15) \cdot 10^{-2}$$

$$\text{Med}(P_{comp}) = 9.97 \cdot 10^{-3}$$

P_{RSS} также усреднено с помощью среднего арифметического - потребление памяти линейно от количества инструкций (остальные накладные расходы пренебрежимо малы). Значит потребление памяти на одну инструкцию также будет в первую очередь зависеть от свойств потоков управления и данных (например от количества ребер на одну инструкцию), но не от размеров трасс. Подобная оценка позволит оценить максимальное суммарное количество инструкций в наборе трасс, который можно обрабатывать одновременно в условиях ограниченной памяти.

$$\overline{P_{RSS}} = \frac{1}{M} \sum_i P_{RSS_i} = (0.443 \pm 0.002) \frac{\Gamma 6}{10^6}$$

Для V приведено взвешенное гармоническое среднее, являющееся средней скоростью обработки всего массива тестовых трасс.

$$\bar{V} = \frac{\sum_i N_i}{\sum_i T_i} = (1.08 \pm 0.04) \cdot 10^5 \frac{1}{\text{с}}$$

3.9 Возможные направления дальнейшего развития решения

В данном разделе будут предложены возможные направления дальнейшего развития решения и новые способы его применения.

3.9.1 Поддержка граничных случаев

Специализированная обработка ряда граничных случаев может существенно повысить качество генерируемых для них модификаций. Примеры возможных оптимизаций представлены далее.

32-битные адресные значения Некоторые среды выполнения на 64-битных платформах используют компактные 32-битные представления ссылок для снижения потребления памяти: подобные представления используются в Android Runtime, HotSpot JVM и V8 [2; 35; 51]. Свободные биты указателей могут использоваться для хранения метаданных [3; 51]. При модификации такого компактного адреса новое значение может не помещаться в исходное представление. В подобных случаях появляются типовые компенсации: выставление нескольких старших бит адреса (часто одинаковых от обращения к обращению). Для таких случаев можно добавлять в трассу специализированные записи, которые позволят компенсировать только эти старшие биты, что может существенно улучшить репрезентативность для ряда сценариев. Подобная оптимизация требует поддержки в обработчиках трасс.

Хеширование адресов Для оптимизации компенсаций хешей адресов можно детектировать хеш функции по символьной информации (в случае полноценного вызова) или с помощью специализированного алгоритма, детектирующего хеширование на ГПД с помощью методов сопоставления сигнатур графов [18; 22], символьного исполнения [27] и SAT решателей [21] (при отсутствии символьной информации или инлайнинге). Значения хешей для детектированных функций можно компенсировать сразу после их вычисления, что позволит избежать распространения некорректных значений по потоку данных и уменьшить количество инструкций приложения, для которых добавляется компенсационный код.

3.9.2 Оптимизация по памяти

В разделе 3.8.3 показано, что большую часть памяти во время обработки трассы занимает ГПД, и что объем памяти, занимаемый графом, линейен по количеству инструкций трассы.

В разделе 3.8.4 приведено усредненное потребление RAM во время тестовых запусков на реальных трассах исполнения заказчика. Подобное потребление не позволяет обрабатывать трассы с количеством инструкций порядка $1 \cdot 10^9$ и более при доступном объеме RAM, характерном для инфраструктуры заказчика. Также высокое потребление памяти существенно ограничивает возможности для параллельной обработки наборов трасс.

Для снижения потребления памяти можно использовать специализированные библиотеки для работы с большими графами [29]. Такие решения хранят графы на диске и подкачивают данные в RAM при необходимости. Также могут быть применены менее специализированные библиотеки: библиотека STXXL [17], предоставляющая реализации для типовых C++ контейнеров, оптимизированных под большие объемы данных; библиотеки LLNL Metall

[33] и LLNL Umap [50], предоставляющие высокоуровневые интерфейсы для выделения памяти через отображаемые файлы (ОС может выгружать такую память на диск) и контроля над выгрузкой страниц соответственно, что позволяет использовать эти библиотеки для ограничения объема используемой RAM.

Другим вариантом решения проблемы является отказ от хранения ГПД для всей трассы. Возможна блочная обработка трассы, со сбором ГПД только для обрабатываемого в данный момент блока. Такой подход потребует многократного сбора одних и тех же участков графа для осуществления всех необходимых обходов графа и обходов трассы, использующих данные графа. Подобное многократное восстановление данных может существенно замедлить обработку трасс, в таком случае потребуются дополнительные оптимизации.

3.9.3 Поддержка векторных регистров

В данный момент на ГПД добавляются только зависимости по памяти и регистрам общего назначения. В итоге поток данных векторных инструкций не отслеживается и не анализируется, хотя в этих данных также могут быть адреса. Более того, некоторые инструкции могут напрямую использовать векторные регистры при обращениях к памяти, например инструкции `gather load` и `scatter store` из расширения SVE архитектуры ARM64 или индексированные векторные загрузки и сохранения из векторного расширения RISC-V [5; 42]. Корректная обработка таких инструкций без отслеживания значений векторных регистров невозможна.

Поддержка векторных регистров может быть добавлена без существенной переработки кодовой базы и алгоритмов. Можно разбивать значения таких регистров на восьмибайтовые блоки и добавлять отдельные ребра для каждого блока, как это уже делается для больших обращений в память.

3.9.4 Модификация защищенных адресов

Представленное решение может быть использовано не только для изменения образов памяти трасс, но и для других задач, требующих поиска адресов в трассе. Одной из подобных задач является обработка адресов, защищенных аппаратными механизмами контроля целостности.

Механизмы защиты потока управления, такие как расширение PAuth архитектуры ARM64 [37; 41] и расширение Zicfiss архитектуры RISC-V [43], позволяют проверять целостность используемых адресных значений. В случае PAuth целостность контролируется с

помощью криптографической подписи адреса и ее проверки непосредственно перед использованием адреса. В случае Zicfiss адреса возврата дополнительно сохраняются в теневом стеке и при возврате сравниваются со значениями, извлеченными из обычного стека. Это позволяет защитить систему от ряда кибер атак, основанных на подмене адресов потока управления.

При анализе трасс, содержащих инструкции подобных расширений, может потребоваться согласованная модификация защищенных адресных значений и связанных с ними метаданных. Для PAuth это может означать переподпись адресов, так как ключ шифрования трассируемой системы неизвестен (и отличается от ключа, используемого симулятором). Для Zicfiss это может означать обновление копий адресов возврата в теневом стеке, указателей на теневой стек и контрольных значений, используемых при переключении стеков. Локации и контексты использования защищенных адресов можно искать с помощью описанных в данной работе алгоритмов после их незначительной доработки.

4 Заключение

В рамках работы было представлено решение для модификации записанных в трассах исполнения образов памяти приложений с целью адаптации трасс для исследования производительности разрабатываемых платформ с новыми настройками системы памяти. Области исходного образа памяти перемещаются для удовлетворения новых ограничений. Генерация необходимых модификаций осуществляется на основании анализа графа потока данных трассы.

Задача разработки решения была успешно решена: разработано представление ГПД, выбраны виды используемых модификаций трассы, построены многопроходные алгоритмы для сбора ГПД и его анализа, реализованы другие вспомогательные компоненты.

Решение было протестировано на наборе целевых трасс заказчика. Выходные трассы были верифицированы с помощью функциональной симуляции. В ходе тестирования были собраны различные метрики для количественной оценки решения и выходных трасс. Для оценки влияния производимых модификаций на репрезентативность выходных трасс были введены метрики N_{comp} и P_{comp} , показывающие, сколько инструкций трассы из-за произведенных модификаций потребуют компенсаций при дальнейшей обработке.

Предложенное графовое представление и алгоритмы позволяют эффективно и с высокой точностью находить в потоке данных трассы адреса и смещения, а также контексты их использования. Симуляционный проход по трассе с применением модификаций позволяет исправлять расхождения потоков управления и данных, хотя и может сильно снижать репрезентативность выходной трассы. В частности, на данном проходе обрабатываются еще не поддерживаемые граничные случаи.

Предложенные метрики для оценки влияния модификаций на репрезентативность трасс собираются во время обработки трассы решением – полноценный анализ трассы на модели не требуется. Это позволяет быстро находить проблемные сценарии.

Была разработана инфраструктура для описания тестовых сценариев для юнит- и модульных тестов. На основе данной инфраструктуры были написаны базовые тесты для модуля сбора ГПД. Возможно применение данной инфраструктуры для покрытия тестами других модулей.

Представленное решение позволяет генерировать консистентные и репрезентативные трассы, удовлетворяющие новым ограничениям системы памяти. 49% тестовых трасс успешно прошли обработку, верификацию и проверку на репрезентативность ($P_{comp} \leq 0.01$). Данные трассы могут быть использованы для исследования производительности разрабатываемых

мой платформы.

Основным научным вкладом автора является разработка методологии для поиска адресов, смещений и контекстов их использования в потоке данных трассы. Также были предложены уже упомянутые в данном разделе метрики и получены экспериментальные данные для количественной оценки реализации.

Разработанное средство успешно применяется для адаптации трасс исполнения под новую конфигурацию системы памяти. Получаемые трассы используются для исследования поведения разрабатываемой системы. Также разработанные алгоритмы и графовое представление могут быть использованы для решения других задач, связанных с поиском адресов в трассах. Например для модификации адресов, защищенных аппаратными механизмами контроля целостности (см. раздел 3.9.4).

К основным ограничениям решения относятся: высокое потребление памяти (линейно по количеству инструкций), проблематичность обработки краевых случаев и вследствие низкая репрезентативность выходных трасс на ряде сценариев. Возможные способы решения этих проблем описаны в разделе 3.9.

Список литературы

1. A gem5 Trace-Driven Simulator for Fast Architecture Exploration of OpenMP Workloads / A. Nocua [и др.] // *Microprocessors and Microsystems*. — 2019. — Т. 67. — С. 42–55. — DOI: 10.1016/j.micpro.2019.01.008.
2. *Android Open Source Project*. Android Runtime: Object Reference Representation. — URL: https://android.googlesource.com/platform/art/+master/runtime/mirror/object_reference.h (дата обр. 04.06.2026).
3. *Android Open Source Project*. Tagged Pointers. — URL: <https://source.android.com/docs/security/test/tagged-pointers> (дата обр. 04.06.2026).
4. *Arm Limited*. Armv8-A Instruction Set Architecture. — 2020. — URL: <https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Learn%20the%20Architecture/Armv8-A%20Instruction%20Set%20Architecture.pdf> (дата обр. 04.06.2026). Arm Developer.
5. *Arm Limited*. Introduction to SVE. — 2020. — URL: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/SVE%20programmers%20guide/102476_0001_00_en_introduction-to-sve.pdf (дата обр. 04.06.2026).
6. *Arm Limited*. Memory Management. — 2021. — URL: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Learn%20the%20Architecture/LearnTheArchitectureMemoryManagement-101811_0100_00_en.pdf (дата обр. 04.06.2026). Arm Developer.
7. Atlantis: Improving the Analysis and Visualization of Large Assembly Execution Traces / H. N. Huang [и др.] // 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). — 2017. — С. 623–627. — DOI: 10.1109/ICSME.2017.23.
8. *Bendersky E.* Position Independent Code (PIC) in shared libraries. — 11.2011. — URL: <https://eli.thegreenplace.net/2011/11/03/position-independent-code-pic-in-shared-libraries/> (дата обр. 16.04.2026). eli.thegreenplace.net.
9. *Bertolino A.* Software Testing Research: Achievements, Challenges, Dreams // 2007 Future of Software Engineering. — 2007. — С. 85–103. — DOI: 10.1109/F0SE.2007.25.
10. *Boost C++ Libraries*. Boost Graph Library. — URL: https://www.boost.org/doc/libs/latest/libs/graph/doc/table_of_contents.html (дата обр. 04.06.2026).
11. *Boost C++ Libraries*. Boost Graph Library: Adjacency List. — URL: https://www.boost.org/doc/libs/latest/libs/graph/doc/adjacency_list.html (дата обр. 04.06.2026).

12. *Boost C++ Libraries*. Boost.Test Documentation. — URL: <https://www.boost.org/doc/libs/latest/libs/test/doc/html/index.html> (дата обр. 04.06.2026).
13. *Catch2 Authors*. Catch2 Documentation. — URL: <https://catch2-temp.readthedocs.io/en/latest/> (дата обр. 04.06.2026).
14. Checkpoint Extraction Using Execution Traces for Intra-task DVFS in Embedded Systems / Т. Tatematsu [и др.] // 2011 Sixth IEEE International Symposium on Electronic Design, Test and Application. — 2011. — С. 19–24. — DOI: 10.1109/DELTA.2011.13.
15. *Chen D., Li D. X., Moseley T.* AutoFDO: Automatic Feedback-Directed Optimization for Warehouse-Scale Applications // Proceedings of the 2016 International Symposium on Code Generation and Optimization. — 2016. — С. 12–23. — DOI: 10.1145/2854038.2854044.
16. *Compilers: Principles, Techniques, and Tools* / А. В. Aho [и др.]. — 2-е изд. — Addison-Wesley, 2007. — ISBN 978-0-321-48681-3.
17. *Dementiev R., Kettner L., Sanders P.* STXXL: Standard Template Library for XXL Data Sets // Software: Practice and Experience. — 2008. — Т. 38, № 6. — С. 589–637. — DOI: 10.1002/spe.844.
18. *Emmert-Streib F., Dehmer M., Shi Y.* Fifty Years of Graph Matching, Network Alignment and Network Comparison // Information Sciences. — 2016. — Т. 346/347. — С. 180–197. — DOI: 10.1016/j.ins.2016.01.074.
19. *Free Software Foundation*. GNU Time: Memory Resources. — URL: https://www.gnu.org/software/time/manual/html_node/Memory-Resources.html (дата обр. 04.06.2026).
20. *Google*. GoogleTest User’s Guide. — URL: <https://google.github.io/googletest/> (дата обр. 04.06.2026).
21. *Handbook of Satisfiability* / под ред. А. Biere [и др.]. — 2-е изд. — IOS Press, 2021. — ISBN 978-1-64368-160-3.
22. *Haq I. U., Caballero J.* A Survey of Binary Code Similarity // ACM Computing Surveys. — 2021. — Т. 54, № 3. — С. 1–38. — DOI: 10.1145/3446371.
23. *Hoffman B., Martin K.* CMake // The Architecture of Open Source Applications. Т. 1. — Lulu.com, 2011. — URL: <https://aosabook.org/en/v1/cmake.html> (дата обр. 04.06.2026).
24. *Introduction to Algorithms* / Т. Н. Cormen [и др.]. — 4-е изд. — MIT Press, 2022. — ISBN 978-0-262-04630-5.

25. *Jones R., Hosking A., Moss E.* The Garbage Collection Handbook: The Art of Automatic Memory Management. — 2-е изд. — CRC Press, 2023. — DOI: 10.1201/9781003276142.
26. *KDE.* Heaptrack: A Heap Memory Profiler for Linux. — URL: <https://github.com/KDE/heaptrack> (дата обр. 04.06.2026).
27. *King J. C.* Symbolic Execution and Program Testing // Communications of the ACM. — 1976. — Т. 19, № 7. — С. 385—394. — DOI: 10.1145/360248.360252.
28. *Kitware.* Testing and CTest. — URL: <https://cmake.org/cmake/help/latest/guide/tutorial/Testing%20and%20CTest.html> (дата обр. 04.06.2026).
29. *Kyrola A., Blelloch G., Guestrin C.* GraphChi: Large-Scale Graph Computation on Just a PC // 10th USENIX Symposium on Operating Systems Design and Implementation. — 2012. — С. 31—46.
30. *Larkin M.* Kernel W^X Improvements in OpenBSD. — 2015. — URL: <https://www.openbsd.org/papers/hackfest2015-w-xor-x.pdf> (дата обр. 04.06.2026). Hackfest 2015.
31. *Levine J. R.* Linkers and Loaders. — Morgan Kaufmann, 2000. — ISBN 978-1-55860-496-4.
32. *Linux Kernel Documentation.* The /proc Filesystem. — URL: <https://docs.kernel.org/filesystems/proc.html> (дата обр. 04.06.2026).
33. Metall: A Persistent Memory Allocator for Data-Centric Analytics / K. Iwabuchi [и др.] // Parallel Computing. — 2022. — Т. 111. — С. 102905. — DOI: 10.1016/j.parco.2022.102905.
34. *Object Management Group.* OMG Unified Modeling Language, Version 2.5.1. — 2017. — URL: <https://www.omg.org/spec/UML/2.5.1/PDF> (дата обр. 04.06.2026).
35. *OpenJDK.* CompressedOops. — URL: <https://wiki.openjdk.org/display/HotSpot/CompressedOops> (дата обр. 04.06.2026).
36. *Orso A., Rothmel G.* Software Testing: A Research Travelogue (2000–2014) // Proceedings of the on Future of Software Engineering. — 2014. — С. 117—132. — DOI: 10.1145/2593882.2593885.
37. PACMAN: Attacking ARM Pointer Authentication with Speculative Execution / J. Ravichandran [и др.] // Proceedings of the 49th Annual International Symposium on Computer Architecture. — 2022. — DOI: 10.1145/3470496.3527429.

38. Pappas V., Polychronakis M., Keromytis A. D. Dynamic Reconstruction of Relocation Information for Stripped Binaries // Research in Attacks, Intrusions and Defenses / под ред. A. Stavrou, H. Bos, G. Portokalidis. — Springer International Publishing, 2014. — С. 68—87. — ISBN 978-3-319-11379-1.
39. PinPlay: A framework for deterministic replay and reproducible analysis of parallel programs / H. Patil [и др.] //. — 04.2010. — С. 2—11. — DOI: 10.1145/1772954.1772958.
40. Prähofer H., Schatz R., Grimmer A. Behavioral model synthesis of PLC programs from execution traces // Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA). — 2014. — С. 1—5. — DOI: 10.1109/ETFA.2014.7005259.
41. Qualcomm Technologies, Inc. Pointer Authentication on ARMv8.3. — 2017. — URL: <https://www.qualcomm.com/media/documents/files/whitepaper-pointer-authentication-on-armv8-3.pdf> (дата обр. 04.06.2026).
42. RISC-V International. “V” Standard Extension for Vector Operations, Version 1.0. — 2025. — URL: <https://docs.riscv.org/reference/isa/unpriv/v-st-ext.html> (дата обр. 11.06.2026).
43. RISC-V International. Control-flow Integrity (CFI). — 2025. — URL: <https://docs.riscv.org/reference/isa/unpriv/unpriv-cfi.html> (дата обр. 11.06.2026).
44. RISC-V International. Cryptography Extensions: Scalar and Entropy Source Instructions, Version 1.0.1. — 2025. — URL: <https://docs.riscv.org/reference/isa/unpriv/scalar-crypto.html> (дата обр. 11.06.2026).
45. RISC-V International. Cryptography Extensions: Vector Instructions, Version 1.0. — 2025. — URL: <https://docs.riscv.org/reference/isa/unpriv/vector-crypto.html> (дата обр. 11.06.2026).
46. Selic B. Tutorial: An Overview of UML 2.0 // Proceedings of the 26th International Conference on Software Engineering. — 2004. — С. 741—742. — DOI: 10.1109/ICSE.2004.1317514.
47. Smith J. E., Nair R. Virtual Machines: Versatile Platforms for Systems and Processes. — Morgan Kaufmann, 2005. — ISBN 978-1-55860-910-5.
48. SynchroTrace: Synchronization-Aware Architecture-Agnostic Traces for Lightweight Multicore Simulation of CMP and HPC Workloads / K. Sangaiah [и др.] // ACM Transactions on Architecture and Code Optimization. — 2018. — Т. 15, № 1. — 2:1—2:26.

49. *Tool Interface Standards Committee*. Executable and Linking Format Specification, Version 1.1. — 1995. — URL: <https://refspecs.linuxfoundation.org/elf/TIS1.1.pdf> (дата обр. 04.06.2026).
50. UMap: Enabling Application-Driven Optimizations for Page Management / I. B. Peng [и др.] // 2019 IEEE/ACM Workshop on Memory Centric High Performance Computing. — 2019. — С. 71—78. — DOI: 10.1109/MCHPC49590.2019.00017.
51. *V8 Team*. Pointer Compression in V8. — 2020. — URL: <https://v8.dev/blog/pointer-compression> (дата обр. 04.06.2026).
52. *Ye H., Hu H.* Too Subtle to Notice: Investigating Executable Stack Issues in Linux Systems // Network and Distributed System Security Symposium. — 2025. — DOI: 10.14722/ndss.2025.230924.
53. *Zhang X., Gupta R.* Whole Execution Traces and Their Applications // ACM Transactions on Architecture and Code Optimization. — 2005. — Т. 2, № 3. — С. 301—334. — DOI: 10.1145/1089008.1089012.
54. *С.А. Л.* Исследование и оптимизация применения трасс исполнения приложения для статической бинарной трансляции под RISC архитектуры : дис. ... канд. / С.А. Лисичин. — ФГАОУ ВО Московский физико-технический институт (национальный исследовательский университет), 2022.